UNIVERSITY OF AMSTERDAM

Interpretable Machine Learning — Rule Generation for Classification: A Comparative Study of RUG in the Binary Setting

Author: Seveah Hung
Student ID: 13591460

BSc Business Analytics

Faculty of Economics and Business

Supervisor: Tabea Röber

26th, June 2025

Abstract

In recent years, the demand for interpretable machine learning has grown, especially in high-stakes domains where transparency and trust are essential. This thesis explores the use case of RUG by evaluating on its performance and interpretability in a binary classification task.

By applying RUG to German Credit dataset, we evaluate RUG against two established glass-box models— CORELS and SLIM — to explore their relative strength in predictive performance and interpretability. Experiments were conducted across various hyperparameter settings. Common evaluation metrics such as accuracy, F1-score, precision, recall, ROC-AUC were used to evaluate the model performance. Moreover, interpretability indicators such as rule length and number of rules were used to assess the model interpretability.

Among three models, SLIM demonstrated the best performance with highest evaluation metrics. CORELS offers the most concise and interpretable results. RUG, while slightly behind in performance, demonstrates strong interpretability through its structure and offers competitive AUC scores. The results shows that RUG provides a compelling balance between accuracy, interpretability, and scalability, making it a promising candidate for real-world decision-making systems.

Contents

1 Introduction **3**

2 Literature Review and Case Background **4**
2.1 Glass-box Models 4
2.2 Scoring Methods: SLIM 5
2.3 Rule Lists: CORELS 5
2.4 Rule Sets: RUG 6
2.5 Case Background 6

3 Methodology **7**
3.1 Datasets 7
3.2 Setup and Modeling 7
3.3 Evaluation 8

4 Results **9**
4.1 RUG 9
4.2 CORELS 10
4.3 SLIM 12

5 Discussion and interpretation of results **12**
5.1 Performance of the models 12
5.2 Interpretation of the models 14
5.3 limitation and future research 17

6 conclusion **17**

1 Introduction

Machine learning (ML) models have increasingly been used to support decision-making in high-stakes domains such as finance, healthcare, and criminal justice. Although high predictive accuracy is achieved, traditional models such as random forests and neural networks often lack interpretability. This prevents us from fully understanding which features are important for our predictions. As such, this kind of "black-box" model hinders trust and adoption.

In real-world decision-making scenarios, interpretability plays a vital role. To bridge this gap, there is a growing demand for interpretable machine-learning approaches, which allow us to understand and interpret the results. By exposing which features drive each prediction, these methods foster a deeper understanding, facilitate model validation, and accelerate trust in high-stakes applications.

XAI methods can further be categorized into two cases. One is the post-hoc explanations where the model is interpreted after training. The other is the intrinsically interpretable model, designed to be easily understood by humans from the start. These models are often referred to as "glass-box" models. Even though glass-box models are simpler by construction, they can still compete with more complex models in many tasks, especially when the data are well-structured tabular data.

To address the need, different methods have been proposed and all share the same goal: being easy to understand. These include rule lists, rule sets, and scoring methods. Although they share a similar goal, they differ in structure and output. Rule sets allow for multiple rules simultaneously, rule lists have ordered rules where only the first rule applies, and scoring methods assign weights to the features.

Among these models, rule-based methods stand out because of their mechanism. Its "if-then" structure makes the results clear and accountable, especially in those fields where clarity is important. However, one key challenge of rule-based methods is scalability, as they often require significant computational resources to search through large combinations of rules. Balancing interpretability and scalability therefore remains an ongoing challenge.

Hence, a very recent rule-based method: Rule Generation for Classification (RUG), has been introduced by Röber et al. (2025). Unlike many earlier approaches that struggle with scalability on large datasets, RUG performs competitively across a wide range of benchmark datasets. The RUG framework uses linear programming with column generation to produce rule sets. These drastically reduce the computational burden, making RUG scalable to large datasets.

The goal of this thesis is to extend RUG by evaluating its performance and interpretability. In this paper, RUG is applied to a binary dataset, and its results are compared with those of two other interpretable methods: a scoring method and a rule list method. This comparison aims to assess the practical usability of RUG and contribute to the broader discussion on how interpretable machine learning methods can be applied in real-world decision-making systems.

The central research question of this thesis is: *How do we best showcase*

the use case of RUG for Binary Classification Problem? The remainder of this paper is organized as follows. In the following section, related literature and theoretical background will be discussed. Then, in section 3, the methodology is described, including the implementation of RUG and baseline models. Section 4 showcases the results, followed by a discussion in section 5.

2 Literature Review and Case Background

2.1 Glass-box Models

As discussed in Adadi and Berrada (2018), interpretable machine learning is a core subfield of XAI aimed at increasing transparency in predictive models, especially in high-stakes domains. In domains such as finance, medicine, and criminal justice, it is essential that the models are understood not only by the data scientist but also the end users. This fosters trust and accountability in the decision-making process.

Glass-box models are interpretable machine learning models whose mechanisms are intrinsically explainable by humans. Unlike traditional models such as deep neutral networks or ensemble methods which need post-hoc tools to be explained, those glass-box models provide clear logic by construction. Those models are widely used in industry settings, common examples are linear regression, decision trees, and rule-based systems. Inherently interpretable models are not only feasible but are even preferable in high-stakes domains (Rudin, 2019). Several studies also show that among these models, logistic regression and decision trees are the most commonly used for credit scoring, as they strike a good balance between performance and transparency (Bücker et al., 2020; Jukna, 2022; Szwabe & Misiorek, 2018).

Miller (2019) gives a straightforward definition: Interpretability is the degree to which a human can understand the cause of a decision. Another good one by Kim et al. (2016) is: Interpretability is the degree to which a human can consistently predict the model's result. Both definitions pointed out interpretability is about how predictable and understandable a model is to humans. The domain experts have been exploring this in two forms : global interpretability, which focuses on understanding the overall decision logic of an ML model; local interpretability, which is understanding specific decisions/predictions made by that model (Du et al., 2018; Zhang et al., 2020). These two concepts are correlated but not always easy to interpret together. While some studies show that better local interpretability often indicates an underlying global structure, Molnar (2022) argues in his work that in a linear model, even when the coefficients can be locally understood, this does not always translate into better global interpretability.

However, glass-box models do not always perform well on high-dimensional data or data with complex relationships. For example, they often underperform deep learning models when processing tasks such as image recognition and speech recognition. Those data are typically non-linear, high-dimensional, and

contain spatial structure, which is a weakness of glass-box models. Nevertheless, glass-box models tend to outperform on structured tabular data, where the information is organized as rows and columns with each feature being clearly specified (Grinsztajn et al., 2022). This makes them more favorable for high-stakes domains, since those data are often well-structured.

For the rest of this section, we will introduce two main families of glass-box models—scoring methods and rule-based systems—along with the corresponding models we will experiment with in this project.

2.2 Scoring Methods: SLIM

Scoring methods are linear classification models that assign points to features based on their predictive contribution, allowing users to compute a final score by adding, subtracting, and multiplying the points. The resulting scorecards can be applied by non-technical users without the necessity of software tools. Therefore, these models are particularly favored in medical field, since they are easy to use and explain. Yet, building those models from data is hard because they have to be accurate, sparse, contain coprime integer coefficients, and satisfy various real-world constraints (Ustun & Rudin, 2015).

To address this challenge, Ustun and Rudin (2015) created a data-driven scoring system called a Supersparse Linear Integer Model (SLIM). SLIM produces sparse linear models with integer coefficients that can be interpreted as point-based scores. Different from traditional models, SLIM effectively selects features by explicitly penalizing model complexity in a transparent and direct way. The model uses an exact 0–1 loss and ℓ_0 -seminorm to apply hard and explicit penalties on both classification error and the number of features used. This avoids fuzzy regularization terms like L1(Lasso) or L2(Ridge), which merely shrink coefficients but may still result in models containing many non-zero weights due to soft penalties (Hastie et al., 2009). SLIM, on the other hand, avoids extensive parameter tuning by offering clear control over sparsity and interpretability, making it well-suited for high-stakes decision-making.

2.3 Rule Lists: CORELS

Rule-based methods are valued for their if-then structure, which makes them highly interpretable and easy to apply. These models typically fall into two main categories: Rule lists and Rule sets.

Rule lists, also known as decision lists, are models which generate ordered sequences of rules (Letham et al., 2018). In these methods, rules are considered one by one, and the first rule that applies determines the prediction. However, constructing rule lists had been an ongoing challenge for decades. Most earlier methods used heuristics or greedy algorithms, hence there were no way to determine whether the generated rule lists are optimal or how far it is from optimal. To address this, Letham et al. (2018) introduced CORELS (Certifiably Optimal RulE ListS) algorithm in 2018. CORELS uses a clever branch-and-bound search

with tight mathematical bounds and pruning strategies to guarantee optimality. This allows it to find interpretable models with provable guarantees.

2.4 Rule Sets: RUG

Rule sets, different from rule lists, consist of unordered collections of rules. In a rule set, all rules are considered simultaneously, and multiple rules may apply to a single instance, as the predictions are often made via a weighted majority vote over all matching rules. This structure allows rule sets to be more flexible than rule lists, particularly in capturing multiple independent patterns in the data. Despite that, rule sets are usually harder to interpret or compute compared to rule lists, as more efforts may be required to visualize and explain the combined effects between rules. Moreover, scalability has been a challenge for existing rule-based methods, especially when applied to thousands of datasets or more complex rule spaces.

RUG algorithm introduced by Röber et al. (2025) is a rule set method that generates interpretable classification models through a formal optimization approach. Its rule-learning algorithm could flexibly incorporate constraints through mathematical programming, which is not straightforward in traditional machine learning (ML) techniques. Additionally, its algorithm utilizes the column generation (CG) procedure to obtain the rules iteratively, similar to what Demiriz et al. (2002) has done in a LP-based learning method, LPBoost. This is particularly efficient when dealing with large datasets, since it avoids enumerating all possible rules upfront. Instead, RUG maintains a small working set of rules and only adds rules that improve the model's loss function the most. This makes RUG scalable to real-world problems with high-dimensional feature spaces, while preserving both interpretability and the ability to incorporate optimization constraints (Röber et al., 2025).

2.5 Case Background

The goal of this project is to explore and enhance the usability of RUG, with a focus on its application in both binary and multi-class classification tasks.

This project compares its performance and usability to two other glass-box models introduced earlier: SLIM and CORELS. The distinct approaches to interpret models between these methods—scoring models, rule lists, and rule sets—allow for a comprehensive case study of their strengths and trade-offs. A rule list method is chosen instead of another rule set method in order to explore how the presence or absence of rule ordering affects model performance and interpretability.

To support this, we extend the open-source `ruleopt` Python package[1] which implements the RUG algorithm. This package enables users to generate optimized rule sets using integer programming, allowing for interpretable and high-utility predictions in tabular classification tasks.

[1] https://github.com/sametcopur/ruleopt

3 Methodology

3.1 Datasets

For this project, we make use of the German credit dataset from the UC Irvine Machine Learning Repository (Dua & Graff, 1992).

The original dataset contains 1000 records, each representing a credit applicant in Germany. Each individual is labeled as either a good or bad credit risk. There are 20 features, of which 7 are numerical and 13 are categorical. The numerical attributes include features such as age, credit amount, and loan duration in months, while the categorical attributes contains variables like credit history, loan purpose, savings account, and present employment duration. The target variable distinguishes between the credit risk good (1) and bad (2).

In preparation for modeling, the numerical variables remained the same and the 13 categorical variables were one-hot encoded into numerical representations. The target variable was recoded so that 0 denotes a bad credit risk and 1 denotes a good credit risk.

3.2 Setup and Modeling

The experiments carried out use Visual Studio Code as the main code editor. To enable compatibility between different modeling frameworks, two different Python environments were implemented. For the RUG framework, Python 3.11.9 and `ruleopt` Python package is used. For benchmarking methods such as SLIM and CORELS, Python 3.10.2 was used together with the `imodels` package(Singh et al., 2021).

All code was executed locally, and virtual environments were used to manage dependencies. Additional libraries such as `pandas`, `numpy`, and `scikit-learn` were employed for data preprocessing, model training, and evaluation.

To model the RUG framework, cross-validation was used to tune hyperparameters. The dataset was first divided into a training set (80%) and a test set (20%). All model selection and tuning were performed on the training data to avoid overfitting. Within this training set, 5-fold cross-validation was used and various maximum depth values were tested, namely $2, 3, 4, 5,$ and 6. This method divides the training data into five equally sized folds, and each fold serves as the validation data in turn for validation.

With regards to CORELS, similar same method was carried out. A 5-fold cross validation was used to tune the parameter `max_depth` before we fit the model.

As for SLIM, we perform 5-fold cross-validation over the parameter `alpha` to identify the optimal one with the highest accuracy. SLIM, short for Supersparse Linear Integer Model, is a scoring-based model that creates linear classification using integer coefficients. Its goal is to create a model that is sparse (with few non-zero coefficients). The associated parameter `alpha` is then to penalize sparsity during optimization, ranging from 0 to 1. A low `alpha` favors more complex models by imposing a weaker penalty. A high `alpha`, in contrast, imposes strong

penalty, resulting in simpler model with potentially lower accuracy. (Ustun & Rudin, 2015) After deciding the optimal `alpha`, it was used to train the final model.

3.3 Evaluation

To assess model performance, we use performance features that are commonly adopted in Machine Learning classification tasks: **precision**, **recall**, **F1-score**, and **accuracy**.

These metrics are derived from the counts of *true positives (TP)*, *true negatives (TN)*, *false positives (FP)*, and *false negatives (FN)*. Each offers insight into different aspects of classification performance.

Precision is defined as:

$$\text{Precision} = \frac{TP}{TP + FP}$$

In our case, it is the proportion of individuals with good credit among those the model predicts to have good credit.

Recall is defined as:

$$\text{Recall} = \frac{TP}{TP + FN}$$

It reflects the proportion of correctly predicted positive instances among all actual positive instances.

F1-score combines precision and recall using the harmonic mean.:

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

It balances the trade-off between precision and recall. It is especially useful when precision and recall are both important for the task.

Accuracy is defined as:

$$\text{Accuracy} = \frac{TP + TN}{\text{Total number of instances}}$$

Accuracy represents the overall correctly predictions for both good and bad credit.

Moreover, the **ROC-AUC (Receiver Operating Characteristic - Area Under the Curve)** score is reported. It is the area under the plot of true positive rate and false positive rate across different threshold values. In credit dataset, it represents the probability that a random good-credit applicant is ranked above a random bad-credit applicant. Thus, the higher the AUC is, the more effectively the model distinguishes between the classes.

Finally, model complexity indicators, such as number of rules (global interpretability) and average rule length (local interpretability) are also considered.

(Sokolova & Lapalme, 2009)

4 Results

4.1 RUG

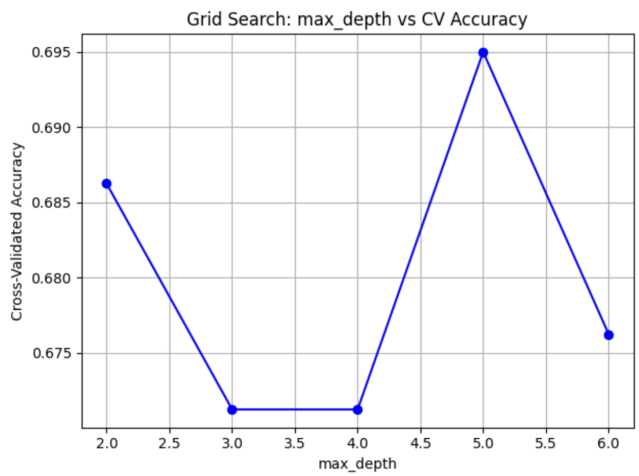


Figure 1: Average accuracy by max_depth for RUG model

Cross-validation To select the optimal configuration for RUG, we perform a grid search over the `max_depth` parameter. The accuracy for several `max_depth` is shown in Figure 1. The highest cross-validated accuracy (0.695) was observed at `max_depth` = 5, and was thus selected to train the final model.

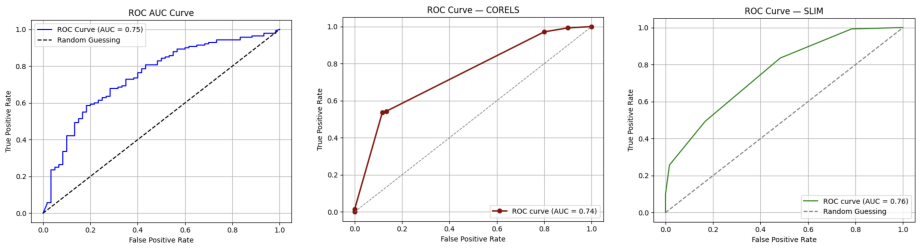
Although `max_depth`=2 provides better interpretability while still obtaining reasonable accuracy (0.686), in very large-scale financial applications even a $0.5 - 1\%$ difference in accuracy can result in significant financial consequences. Additionally, prior study (Lakkaraju et al., 2016) has shown that for classification tasks, rules with $3-5$ conditions are typically considered as interpretable by most humans, especially when the rule-based method has its unique IF–THEN structure.

Therefore, to balance accuracy and interpretability, the `max_depth`=5 was chosen to train the final model. The final evaluation metrics were computed in the held-out test set for each `max_depth`, as shown in table 1.

Results on test set Table 1 presents the performance on the test set across different `max_depth` values. As shown, `max_depth` = 3 yields the best overall performance, with the highest accuracy (0.72), precision(0.68), recall(0.69), and F1-score(0.68). `max_depth`=4 is the second best, obtaining comparable performance with each metric being 1% lower than that for `max_depth`=5. In comparison, `max_depth`=2 performed the worst across all metrics. Lastly, `max_depth` of 5 and 6 showed moderate results, though `max_depth`=5 slightly outperformed depth 6 in both precision (0.65 vs. 0.63) and recall(0.67 vs. 0.63).

max_depth	Precision	Recall	F1-Score	Accuracy
2	0.62	0.64	0.63	0.66
3	**0.68**	**0.69**	**0.68**	**0.72**
4	0.67	0.68	0.67	0.71
5	0.65	0.67	0.66	0.69
6	0.63	0.63	0.63	0.69

Table 1: RUG classifier performance on the test set for different values of `max_depth`. Credit data.



(a) RUG (b) CORELS (c) SLIM

Figure 2: ROC AUC curve

Additionally, the ROC curve in Figure 2a shows that RUG achieves an AUC of 0.75. This means that the classifier ranks good-credit applicants higher than bad-credit applicants 75% of the time, indicating a solid level of discriminative ability.

Lastly, based on the code output, the trained RUG model produced 131 rules. This is the total number of rules used in the model. On average, each rule contains approximately 4.05 conditions, which supports interpretability. For each test observation, an average of 6.88 rules apply, and these rules have an average length of 3.92 conditions.

4.2 CORELS

Cross-validation Similar to what was performed in RUG, the average accuracy results from a 5 fold cross validation for CORELS are presented in Figure 3. According to the graph, `max_depth`=2 achieved the best average accuracy during the validation. Hence, the parameter was selected to train the final model.

Results on test set With regards to Table 2, the CORELS shows fairly consistent accuracy across different values of `max_depth`. Notably, recall is much higher than precision across all configurations; the difference is more than 20%. This indicates that the model is more effective at identifying positive cases than

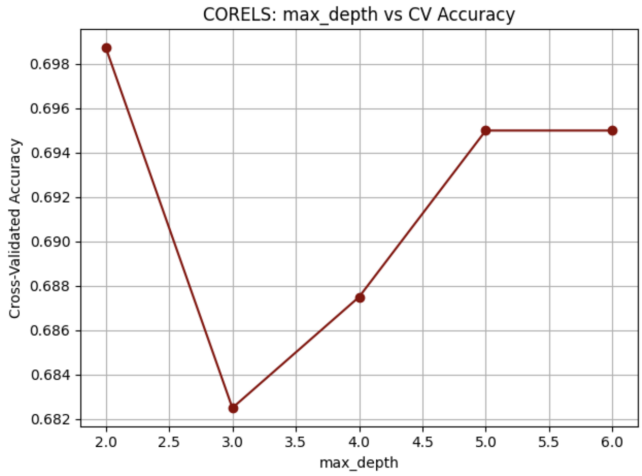


Figure 3: Average accuracy by max_depth for CORELS model

avoiding false positives.

Additionally, the ROC curve in Figure 2b shows that CORELS achieves an AUC of 0.74, indicating a strong discriminative ability between two groups.

Lastly, the final CORELS model produces a rule list with only 2 rules; this makes it a model easily understood.

Table 2: Performance of CORELS at varying maximum rule list depths. Credit data.

max_depth	**Precision**	**Recall**	**F1 Score**	**Accuracy**
2	0.74	0.95	0.83	0.73
3	0.74	0.96	0.84	0.74
4	**0.74**	**0.97**	**0.84**	**0.74**
5	0.74	0.97	0.84	0.74
6	0.74	0.97	0.84	0.74

4.3 SLIM

Table 3: Cross-validated accuracy of SLIM at different sparsity weights (α).

Alpha (α)	**CV Accuracy**
0.10	0.6925
0.30	0.6963
0.50	0.6963
0.70	**0.7038**
1.00	0.7025

Cross-validation During the cross-validation, several values of `alpha` have been tried out, namely 0.1, 0.3, 0.5, 0.7, and 1.0. According to Table 3, an `alpha` value of 0.7 obtained the highest average accuracy at 0.7038. This value was selected for our final model. The test result is reported below.

Table 4: Performance of SLIM at varying sparsity weights (α). Credit data.

Alpha (α)	**Precision**	**Recall**	**F1 Score**	**Accuracy**
0.10	0.747	0.993	0.853	0.760
0.30	0.768	0.829	0.797	0.705
0.50	0.747	0.993	0.853	0.760
0.70	0.747	0.993	0.853	0.760
1.00	0.747	0.993	0.853	0.760

Results on test set In Table 4, we observe a fairly consistent performance across different values for `alpha`. Similar to CORELS, SLIM tends to achieve higher recall than precision, with most recall values approaching 1. This also means that SLIM almost never misclassifies good applicants, but is more likely to produce false positive. Nonetheless, SLIM shows the best overall performance across all evaluation metrics.

Furthermore, the ROC curve in Figure 2c shows that SLIM achieves an AUC of 0.76. This is similar to how RUG and CORELS perform, with AUCs of 0.75 and 0.74 respectively.

5 Discussion and interpretation of results

5.1 Performance of the models

RUG According to the results presented in table 1, the RUG model performs well on the unseen credit data. A maximum depth of 3 achieves the best performance, with all metrics indicators being the highest values. In total, 72%

of the applicants were correctly classified. This configuration demonstrates an optimal trade-off: It offers the best predictive power while keeping the rules interpretable.

Moreover, the model shows relatively stable performance when `max_depth` increases by one level. This pattern reflects that RUG is somewhat robust to small changes in hyperparameters. However, as `max_depth` increases further, precision and recall start to drop more noticeably, even though accuracy remains relatively stable. In the context of credit scoring, this suggests more false positive. Misclassifying more high-risk applicants as creditworthy can lead to significant financial risk. From a fairness perspective, although precision seems to drop faster than recall when `max_depth` differs from 3, the difference remains within 2%. This suggests that RUG demonstrates fairness for individuals with good and bad credit; hence no major fairness concerns are raised.

RUG's AUC of 0.75, as illustrated in Figure 2a, confirms that the model is good in distinguishing between good and bad credit classes. In risk modeling, an AUC above 0.7 is generally considered acceptable, while values exceeding 0.8 are strong (Ling & Huang, 2003). With an AUC of 0.75, RUG demonstrates a meaningful discriminatory power that aligns well with industry benchmark. This again shows its value in decision-making.

CORELS For CORELS's performance on credit classification task, as shown earlier, the model yields stable performance across `max_depth` from 3 to 6. For `max_depth`of 3 and 4, the differences of accuracy are minor. Accuracy for CORELS remains around 0.74, being slightly better than RUG's. However, with a maximum depth of 2, CORELS significantly outperform RUG in terms of all metrics, with accuracy of 0.73 compared to RUG's 0.66.

One of the most notable aspects of COREL's performance is its consistently high recall across all configurations. Ranging from 0.95 to 0.97, CORELS excels at identifying good credit applicants. In contrast, its ability to detect bad credit applicants is way weaker. Despite the trade-off between recall and precision, CORELS maintains a relatively high precision of around 0.74. In credit scoring contexts, this is still a promising result.

Finally, Figure 2b presents an AUC of 0.74 for the final CORELS model. This indicates a solid overall ability to distinguish between two classes. Compared to the ROC curve for RUG, CORELS shows a more step-like structure. This is likely due to the rule-list mechanism of CORELS, where each time only the first rule matched determines the prediction. While having different structure, the two models demonstrate a similar ability to distinguish between good and bed credit applicants.

SLIM As shown earlier in the Table 4, SLIM seems to achieve the best overall accuracy across and is the most stable among all models. Interestingly, it yields identical performance metrics expect for $\alpha = 0.3$. This may suggest that the trained SLIM model is already sparse; an stronger penalty (e.g., to 0.7 or 1.0) does not further increase the accuracy. Besides, SLIM may drop one or more

features for predicting class 1 at αvalue of 0.3, leading to a lower recall. These results indicates that SLIM is robust to changes in α across a wide range and consistently prefer high recall over precision. In comparison, RUG offers more balanced in precision and recall, but its best accuracy (0.71) is lower than SLIM's (0.76), and its F1 score (0.68) is also worse than slim's (0.853). Furthermore, in terns of AUC , the three models demonstrate similar ability in distinguishing between good and bad credit applicants. Overall, SLIM outperforms both RUG and CORELS in terms of F1 score and accuracy, particularly excelling in recall.

5.2 Interpretation of the models

RUG As this thesis focuses on interpretable machine learning, it is essential to access not only the model performance but how easy can the decision-making can be understood. At the global level, the trained RUG demonstrates its global complexity by producing a total of 131 rules, with an average of 4.05 conditions per rule generated. As discussed earlier, this falls within a range generally considered easy to interpret. In terms of local interpretability, each test observation is covered by an average of 6.88 rules, and these rules have an average length of 3.92 conditions. All these qualities suggest that RUG may be a suitable choice for credit scoring applications, where model transparency plays an important role.

To further illustrate local interpretability, we examine the model's prediction for the first individual from the test set. In this observation, a total of 8 rules apply, referring to Table 5.

Among them, Rule 83 predicts a bad credit with a weight of 0.1276, based on four conditions. Specifically, the applicant has a credit amount between 2198 and 4158, does not have a bank-based payment plan, is older than 22.5 years, and with at least 200 Deutsche Marks or receives regular salary deposits. This rule 83 appears reasonable, since it is possible that a person in their 20s take out a mid-sized loan with an additional payment plan could not pay it back. The condition with age being greater than 22.5 may seems odd at the beginning, but it is not necessary older individuals have a bad credit. Instead, older applicants (say, over 55) with the same features could be fewer or excluded by other rules. The last condition is relatively on the positive side, but since all the conditions has to be satisfied simultaneously, the impact of it may be outweighed by other conditions. In this case, the condition with no backup payment plan may be the one that carries more weight.

In comparison, Rule 90 supports a class 1 prediction with a weight of 0.1116. The credit amount in this case is much smaller than that in rule 83. An individual taking out a small-sized loan with duration greater then 16.5 may indicates their financial stability. These are the characteristics typically associated with good creditworthiness.

By summing over the weighted contributions of these rules, the class 0 obtains a total of 0.3446 and class 1 receives a weight of 0.1489. Therefore, RUG makes a final prediction of class 0 ($0.3446 > 0.1489$).

Overall, these results highlights RUG not only demonstrates strong predictive performance but enables transparent and actionable explanations. It offers results comparable to more complex models while maintaining a high level of rule simplicity. This makes RUG a particularly suitable candidate model for high-stakes domain, where both prediction quality and explainability are critical.

Table 5: Applicable rules for the 1st test observation in RUG for credit risk classification

Rule ID	**Label**	**Weight**	**Conditions**
83	0	0.1276	2198.0 < credit_amount < 4158.0 AND other_payment_plans_1 < 0.5 AND 22.5 < age AND 0.5 < checking_status_3
90	1	0.1116	2254.0 < credit_amount < 2393.0 AND 16.5 < duration
93	0	0.1008	0.5 < savings_status_1 AND employment_2 < 0.5 AND purpose_2 < 0.5 AND credit_amount < 8559.5
98	0	0.0799	age < 47.0 AND employment_4 < 0.5 AND residence_since < 2.5 AND purpose_8 < 0.5 AND purpose_9 < 0.5
120	0	0.0243	other_payment_plans_1 < 0.5 AND 25.5 < duration AND employment_1 < 0.5 AND job_1 < 0.5 AND 29.5 < age
123	1	0.0192	25.5 < age AND credit_amount < 3886.0 AND credit_history_4 < 0.5 AND 0.5 < foreign_worker_1
124	1	0.0181	1035.0 < credit_amount < 8372.0 AND age < 53.5
126	0	0.0120	savings_status_3 < 0.5 AND 22.5 < age AND 20.5 < duration AND 2.5 < installment_commitment AND credit_history_1 < 0.5

CORELS To examine the interpretability, we inspect the rule list CORELS produces for the credit dataset.

Table 6: Final rule list learned by CORELS on the credit dataset

Rule #	**Condition**	**Predicted Class (prob)**	**Applies to (# samples)**
1	`checking_status_3` > 0.5	Class 1 (0.876)	314
2	`duration` ≤ 31.5	Class 1 (0.634)	391

As displayed in table 6, an order rule list is shown. Since `max_depth` was set to 2, the list contains only two rules. The first rule states that if the feature

checking status_3 is greater than 0.5, predict good credit. Since this feature is binary, this can read as: if checking status_3 is equal to 1, predict class 1, with a probability of 0.876. Among 800 observations, 314 samples meet this condition and are classified as good credit. Those that failed to satisfy this condition proceed to the next rule. The second rule assigns class 1 to those whose loan duration is less than or equal to 31.5 months, with a lower probability of 0.634. This rule list aligns with intuition, since individuals with at least 200 Deutsche Marks or regular salary deposit tend to be creditworthy; those with shorter loan durations pose lower risk.

Overall, together with the performance metrics reported earlier, CORELS showcases strong classification ability while maintaining high interpretability. The learned rule list generated is short and aligned with financial reasoning, making CORELS a potentially more suitable model for credit scoring compared to RUG.

Table 7: Non-zero coefficients from the SLIM model on the credit dataset.

Feature	**Original Meaning**	**Coefficient**
checking_status_1	status < 0 DM	-1
checking_status_3	≥ 200 DM or salary assignment	1
credit_history_1	no credits/all paid duly	1
other_payment_plans_1	bank	1
purpose_4	domestic appliances	-1
purpose_9	business	1
savings_status_4	≥ 1000 DM	1

SLIM SLIM is a sparse linear model with integer coefficients. To illustrate the interpretability, we printed out the non-zero coefficients in credit dataset.(see Table 7)

To explain the local interpretability, we take the first sample in the test set as an example:

Table 8: SLIM Score Calculation for the First Test Sample

Feature	**Original Meaning**	**Coefficient**
Intercept	—	0.2154
`checking_status_3`	≥ 200 DM or salary assignment	+1
`credit_history_1`	No credits / all paid duly	+1

The total score is calculated by summing the intercept and the active features:

$$\text{Score} = 0.2154 + (1 \times 1) + (1 \times 1) = 2.2154$$

Using the logistic function, the predicted probability is obtained as:

$$\text{Probability} = \frac{1}{1 + e^{-2.2154}} \approx 0.9016$$

This result indicates this applicant is likely to be classified as a good credit (class 1). When verifying, we found this aligns with the actual prediction. Notably, the first rule was also used in CORELS example, suggesting that this is a rule commonly shared across models. Indeed, an individual with regular salary deposit or higher DM is more creditworthy. Having no outstanding credits or all payment made duly is also an indicator for financial stability. Hence the logic is quite reasonable for human interpretation. In this case, SLIM makes prediction based on just two rules, reflecting some level of simplicity.

5.3 limitation and future research

This project carried out the binary classification task for the three models. Hence,future research can further explore the multi-class classification, which was not fully explored within the scope of this study. One potential limitation may be within SLIM. Despite having strong performance, it only applies to binary classification by construction. Further extension on this remain an open area for research.

6 conclusion

This thesis investigated the interpretability and performance of RUG in comparison with two widely studied interpretable methods—CORELS (rule list) and SLIM (scoring method)—across binary classification tasks. Through extensive experiments on the credit dataset, we evaluated not only predictive accuracy but also the practical interpretability of each model.

Overall, SLIM offers a good trade-off in terms of accuracy and interpretability. Nonetheless, its mechanism is not as straightforward as other two rule-based methods. CORELS and RUG have better global interpretability but sacrifices mildly in their accuracy compared to SLIM. Among the two rule-based methods, CORELS outperform RUG in both interpretability and accuracy.

RUG, while slightly less accurate, stands out for its flexibility and strong local explanations through aggregated rule-based predictions. Overall, RUG demonstrates a compelling balance between scalability, interpretability, and predictive power. Its structure supports transparent, actionable explanations that are particularly valuable in high-stakes settings such as credit scoring. These results suggest that RUG is a promising candidate for interpretable classification tasks and warrants further development and application.

Acknowledgements

The author is grateful to Tabea Röber, the supervisor of this thesis, for her invaluable support and guidance throughout the project.

References

Adadi, A., & Berrada, M. (2018). Peeking inside the black-box: A survey on explainable artificial intelligence (xai). *IEEE Access*, *6*, 52138–52160.

Bücker, B., Kehlbeck, R., & Wangenheim, F. (2020). Transparency, auditability and explainability of machine learning models in credit scoring. *arXiv preprint arXiv:2009.13384*. https://arxiv.org/abs/2009.13384

Demiriz, A., Bennett, K. P., & Shawe-Taylor, J. (2002). Linear programming boosting via column generation. *Machine Learning*, *46*(1–3), 225–254. https://doi.org/10.1023/A:1012470815092

Du, M., Liu, N., & Hu, X. (2018). Techniques for interpretable machine learning. https://arxiv.org/abs/1808.00033

Dua, D., & Graff, C. (1992). Uci machine learning repository: Statlog (german credit data) [Accessed: 2025-06-22]. https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data

Grinsztajn, L., Oyallon, É., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on tabular data? *Proceedings of the NeurIPS 2022 Track on Datasets and Benchmarks*. https://arxiv.org/abs/2207.08815

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. https://web.stanford.edu/~hastie/ElemStatLearn/

Jukna, V. (2022). Credit scoring methods comparison – a review. *Available at ResearchGate*. https://www.researchgate.net/publication/361254078_Credit_Scoring_Methods_Comparison_-_A_Review

Kim, B., Khanna, R., & Koyejo, O. O. (2016). Examples are not enough, learn to criticize! criticism for interpretability. *Advances in Neural Information Processing Systems*, *29*. https://proceedings.neurips.cc/paper_files/paper/2016/file/a486cd07e4ac3d270571622f4f316ec5-Paper.pdf

Lakkaraju, H., Bach, S., & Leskovec, J. (2016). Interpretable decision sets: A joint framework for description and prediction. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 1675–1684. https://doi.org/10.1145/2939672.2939874

Letham, B., Rudin, C., McCormick, T. H., & Madigan, D. (2018). Interpretable classifiers using rules and boolean combinations: Learning certifiably optimal rule lists. *Journal of Machine Learning Research*, *18*(1), 1–78. https://www.jmlr.org/papers/volume18/17-716/17-716.pdf

Ling, C. X., & Huang, J. (2003). Auc: A better measure than accuracy in comparing learning algorithms. *Proceedings of the Canadian AI Conference*.

Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. *Artificial Intelligence*, *267*, 1–38. https://doi.org/10.1016/j.artint.2018.07.007

Molnar, C. (2022). *Interpretable machine learning* [https://christophm.github.io/interpretable-ml-book/].

Röber, T. E., Lumadjeng, A. C., Akyüz, M. H., & Birbil, Ş. İ. (2025). Rule generation for classification: Scalability, interpretability, and fairness. *Pattern Recognition*. https://doi.org/10.1016/j.patcog.2025.109879

Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, *1*(5), 206–215. https://doi.org/10.1038/s42256-019-0048-x

Singh, C., Nasseri, K., Tan, Y. S., Tang, T., & Yu, B. (2021). *Imodels: A python package for fitting interpretable models* (No. *61*). The Open Journal. https://doi.org/10.21105/joss.03192

Szwabe, A., & Misiorek, A. (2018). Decision trees as interpretable bank credit scoring models. *Information Systems Architecture and Technology: Proceedings of 38th International Conference on Information Systems Architecture and Technology–ISAT 2017, Part I*, 181–191. https://doi.org/10.1007/978-3-319-99987-6_16

Ustun, B., & Rudin, C. (2015). Supersparse linear integer models for optimized medical scoring systems. *Machine Learning*, *102*(3), 349–391. https://doi.org/10.1007/s10994-015-5528-6

Zhang, Q., Hua, Y., Liu, P., Liu, Z., Zhang, X., & Qiu, X. (2020). A survey on neural network interpretability. https://arxiv.org/abs/2012.14261